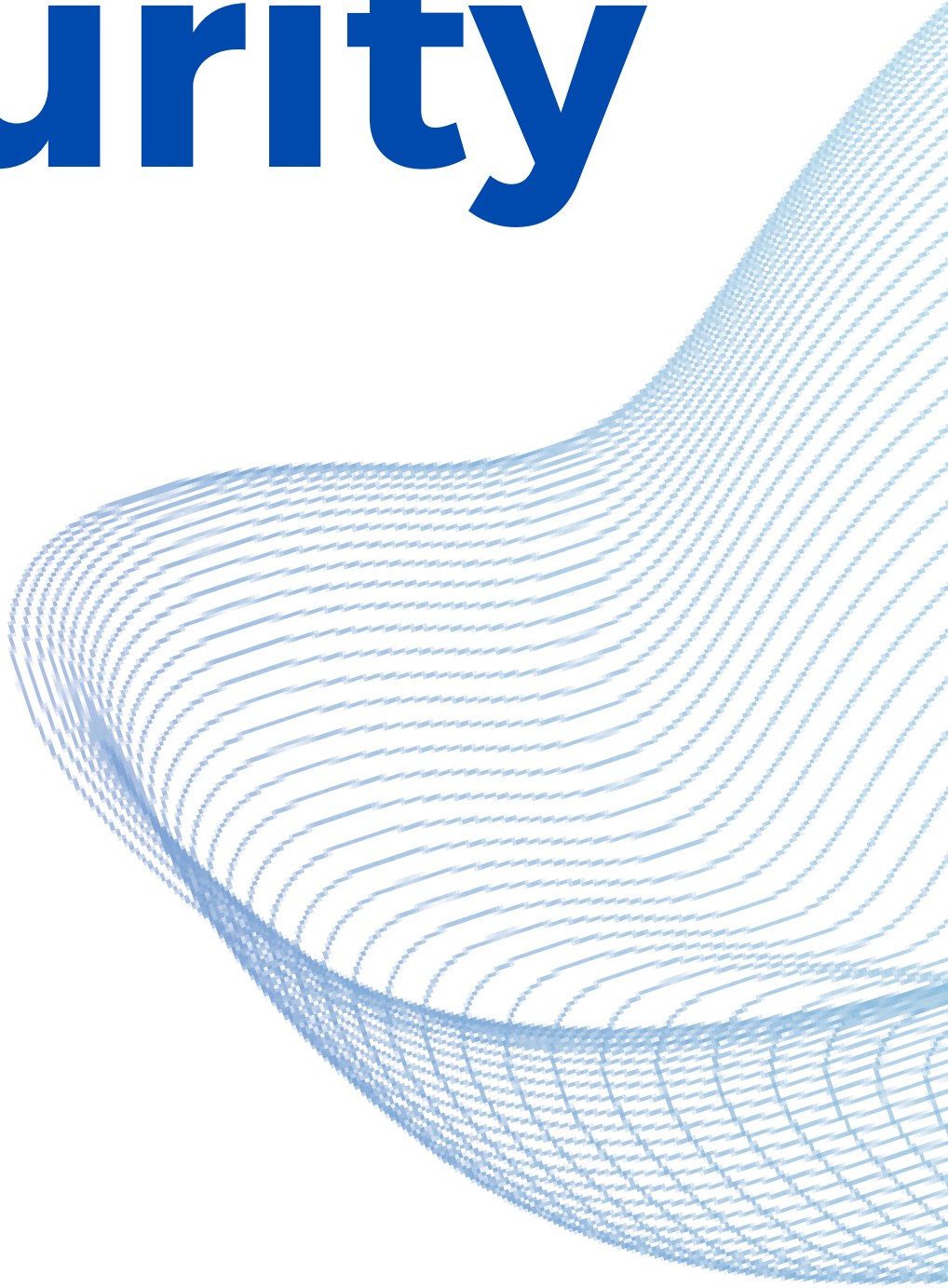


Information Security



Unit- 3 Message Authentication & Hash Function

Message Authentication

- Message authentication is a mechanism or service used to verify the integrity of a message.
- It assures that data received are exactly as sent(i.e., there is no modification, deletion, or replay).
- Message authentication ensures that the message has been sent by a genuine identity and not by an imposter.
- Message authentication is concerned with:
 - Protecting the integrity of message
 - Validating identity of originator
 - Non-repudiation of origin

Message Authentication Function

- Any message authentication mechanism has two levels of functionality.
- At lower level, there must be some sort of function that produces and authenticator: a value to be used to authenticate a message.
- This lower level function is then used as primitive in higher-level authentication protocol that enables a receiver to verify the authenticity of a message.
- There are three types of functions that may be used to produce an authenticator.

1. Hash Function :

- A function that maps a message of any length into a fixed-length hash value, which serves as the authenticator.

2. Message Encryption :

- The ciphertext of the entire message serves as its authenticator.

3. Message Authentication Code :

- A function of the message and a secret key that produces a fixed-length value that serves as the authenticator.

Message Authentication Code (MAC)

- Message Authentication Code is a function of message and a secret key that produces a fixed-length value that serves as the authenticator.
- This technique assumes that the sender and receiver share a common secret key K. When a sender has a message to send receiver, it calculates the MAC as a function of the message and key.

$$\text{MAC} = C(M, K)$$

Where, C = MAC function

M = Input message,

K= Shared key

MAC = Message Authentication Code.

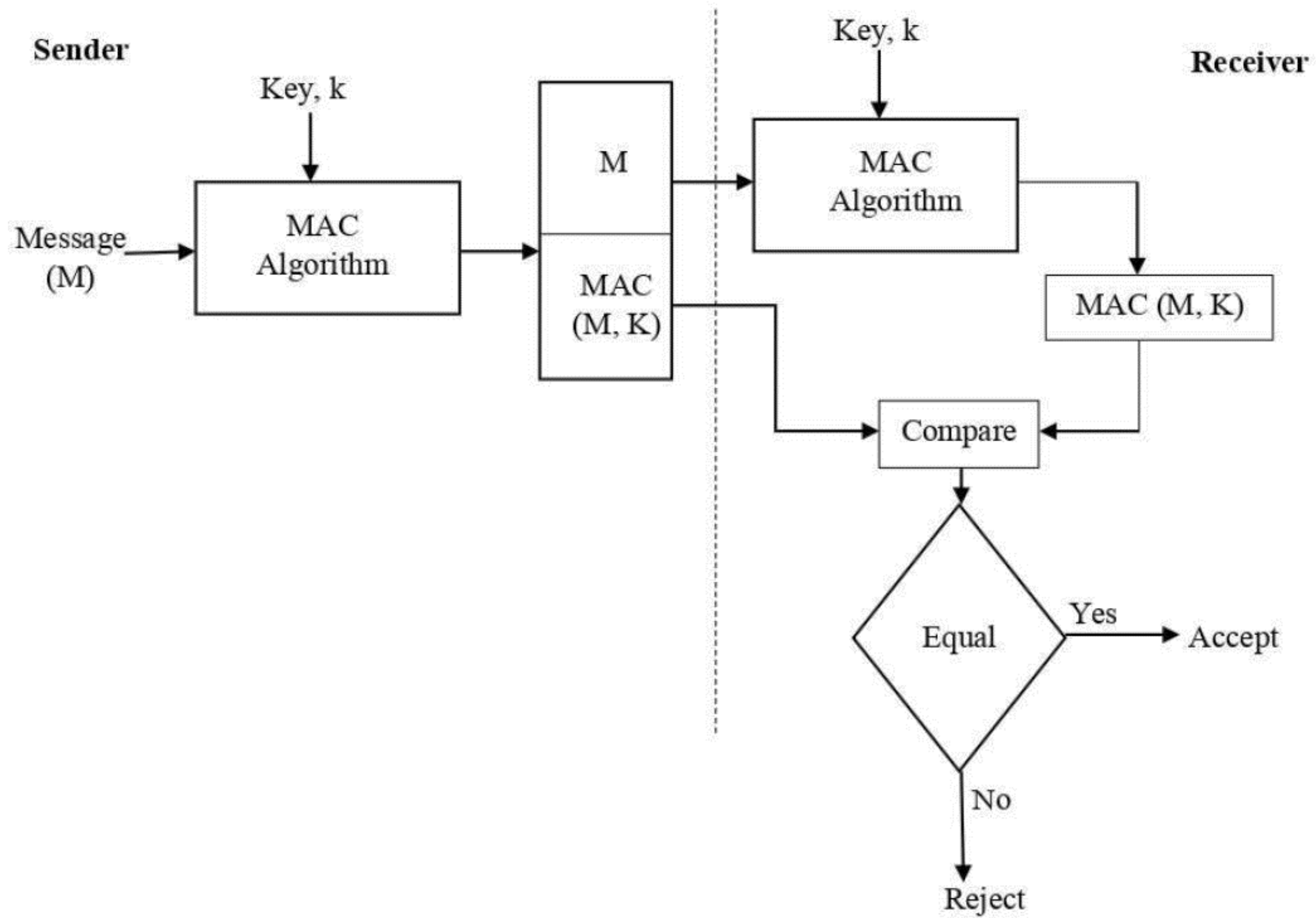


Fig: Message Authentication using a MAC

- The message plus MAC are transmitted to the intended receiver. The receiver performs some calculation on received message using the same secret key to generate a new MAC. Then the received MAC is compared to the calculated.
- If the received MAC matches the calculated MAC then the receiver is assumed that the message has not been altered.
- A MAC function is similar to encryption. One difference is that the MAC algorithm need not be reversible, as it must be for decryption. Because of mathematical properties of the authentication function, it is less vulnerable to being broken than encryption.

Hashing and Hash Function

➤ A hashing family is a 4- tuple (X, Y, K, H) Where the following conditions are satisfied :

X → is a finite set of possible messages

Y → is a finite set of possible message digest

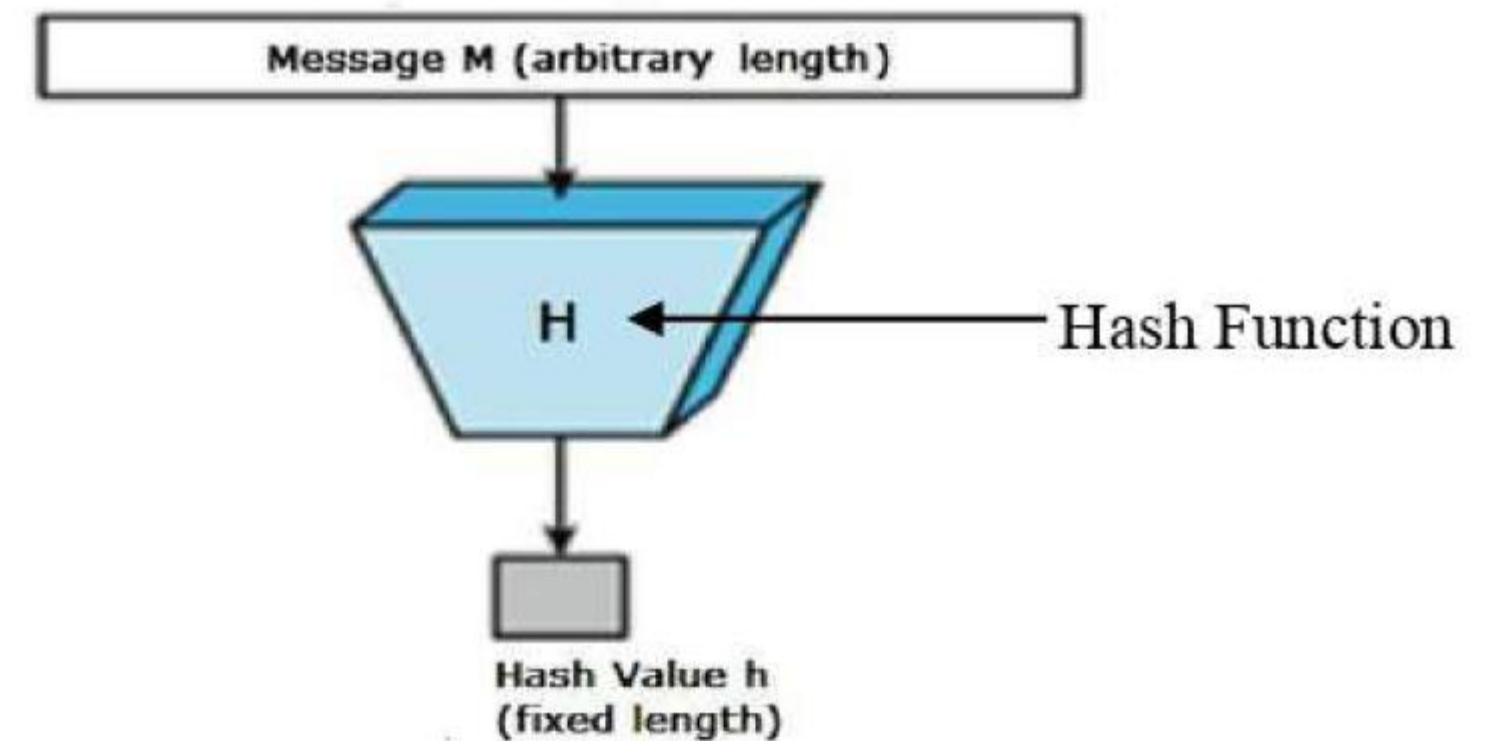
K → the key space is the finite set of possible key

H → For each $k \in K$ there is a hash function $h_k \in H$ such that

$$h_k : X \rightarrow Y$$

➤ A hash function maps a variable length message in to a fixed length hash value or message digest.

➤ The output is unique to the input, meaning even a small change in the input will produce a significantly different output.



Properties of Hash Function

- In order to be an effective cryptographic tool, the hash function is desired to posses following properties-

1. Pre-Image Resistance :

- This property means that it should be computationally hard to find the input (message) from the output (hash value) i.e., for any given hash value h , it should be difficult to find any message m such that $H(m) = h$.

2. Second Pre-image Resistance :

- This property means given an input and its hash, it should be hard to find a different input with the same hash i.e. for any given input m_1 , it is computationally hard to find a different input m_2 such that $H(m_1) = H(m_2)$.
- Also known as weak collision resistance.

Properties of Hash Function

3. Collision Resistance :

- Collision of the hash function is the event whose two values m_1 & m_2 s.t $m_1 \neq m_2$ hash to the same value i.e. $h(m_1) = h(m_2)$. A given hash function is said to have property of collision resistance where it is difficult for the attacker to find the collision.

Applications of Hash Function

1. Message Authentication :

2. Digital Signature :

3. Password Security :

4. Encryption :

5. Message Digest :

Message Digest

- A message digest is a fixed-size numerical representation of the contents of a message, generated by a hash function.

Message Digest Version 4 (MD4)

- The MD4 function is a cryptographic algorithm that takes a message of arbitrary length as input and produces a 128 bit message digest or hash value as output.
- Suppose a b - bit message is a input and we have to find its message digest. It is assumed that the bits of the message are $m_0, m_1, \dots, m_{b-1}$.

Feature	Value
Input Size	Any Length
Output Size	128 bits
Digest Length	16 Bytes
Number of Rounds	3
Block Size	512 bits

1. Append Padded Bits :

- The message is padded so that its length is congruent to 448 , mod 512. A single '1' bit is append to the message and then '0' bits are appended so that the length of bits equals 448 modulo 512.

Message	100000	
---------	--------	--

$$(\text{Message length} + \text{padded bits}) \% 512 = 448$$

2. Append Length :

- A 64-bit representation of b is appended to the result of the previous step. The resulting message has a length that is an exact multiple of 512 bits.
- Divide it to N words as $M_0, M_1, \dots, M_{N-1}$ (N is a multiple of 16).

Message	100000	64 bits
---------	--------	---------

$$(\text{Message length} + \text{padded bits} + 64 \text{ bits}) \% 512 = 0$$

3. Initialize MD Buffer :

- A 4-word buffer (A, B, C, D) is used to compute the message digest. Here each of A, B, C, D is a 32-bit register. These are initialized to the following values in hexadecimal, low order byte first:

A: 01 23 45 67

B : 89 ab cd ef

C : fe dc ba 98

D: 76 54 32 10

And save as A^0 , B^0 , C^0 , D^0

4. Process message in 16-word (512 bit) blocks :

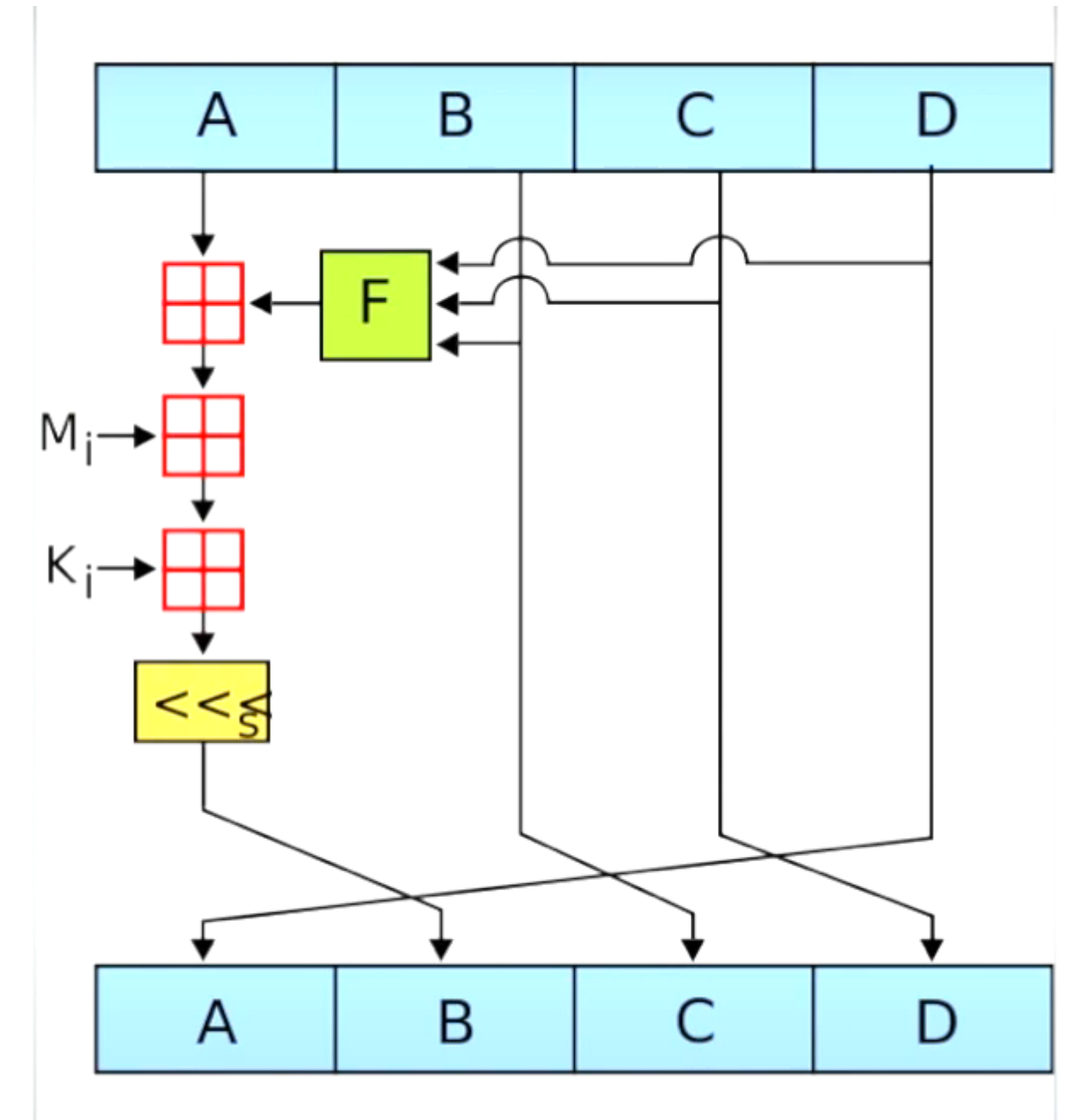
Processing 48 rounds

- 16 rounds
 - $F(B, C, D) = BC \vee (\neg B)D$
 - $i = 0, 1, \dots, 15$
 - $K_i = 0$
 - $s = 3, 7, 11, 19, 3, 7, 11, 19, 3, 7, 11, 19$
- 16 rounds
 - $F(B, C, D) = BC \vee BD \vee CD$
 - $i = 0, 4, 8, 12, 1, 5, 9, 13, 2, 6, 10, 14, 3, 7, 11, 15$
 - $K_i = 5A827999$
 - $s = 3, 5, 9, 13, 3, 5, 9, 13, 3, 5, 9, 13, 3, 5, 9, 13$
- 16 rounds
 - $F(B, C, D) = B \oplus C \oplus D$
 - $i = 0, 8, 4, 12, 2, 10, 6, 14, 1, 9, 5, 13, 3, 11, 7, 15$
 - $K_i = 6ED9EBA1$
 - $s = 3, 9, 11, 15, 3, 9, 11, 15, 3, 9, 11, 15, 3, 9, 11, 15$

Compute

- $A = A + A^0$
- $B = B + B^0$
- $C = C + C^0$
- $D = D + D^0$

and output $A B C D$ as the result



Message Digest Version 5 (MD5)

- The MD5 algorithm, improved version of MD4, is a cryptographic algorithm that takes a message of arbitrary length as input and produces a 128 bit message digest or hash value as output.
- MD5 was invented by Ron Rivest as an improved version of MD4.
- Operations in MD5 are as follows:

1. Append Padded Bits :

- The message is padded so that its length is congruent to 448 , mod 512. A single '1' bit is append to the message and then '0' bits are appended so that the length of bits equals 448 modulo 512.

Message	100000	
---------	--------	--

$$(\text{Message length} + \text{padded bits}) \% 512 = 448$$

2. Append Length :

- A 64-bit representation of b is appended to the result of the previous step. The resulting message has a length that is an exact multiple of 512 bits.
- Divide it to N words as $M_0, M_1, \dots, M_{N-1}$ (N is a multiple of 16).

Message	100000	64 bits
---------	--------	---------

$$(\text{Message length} + \text{padded bits} + 64 \text{ bits}) \% 512 = 0$$

3. Initialize MD Buffer :

- A 4-word buffer (A, B, C, D) is used to compute the message digest. Here each of A, B, C, D is a 32-bit register. These are initialized to the following values in hexadecimal, low order byte first:

A: 01 23 45 67

B : 89 ab cd ef

C : fe dc ba 98

D: 76 54 32 10

And save as A^0 , B^0 , C^0 , D^0

4. Process message in 16-word (512 bit) blocks :

- It contain 4-passes (rounds) with 16 steps or operation each.
- We first define four functions that each take as input three 32-bit words and produce as output one 32-bit word.

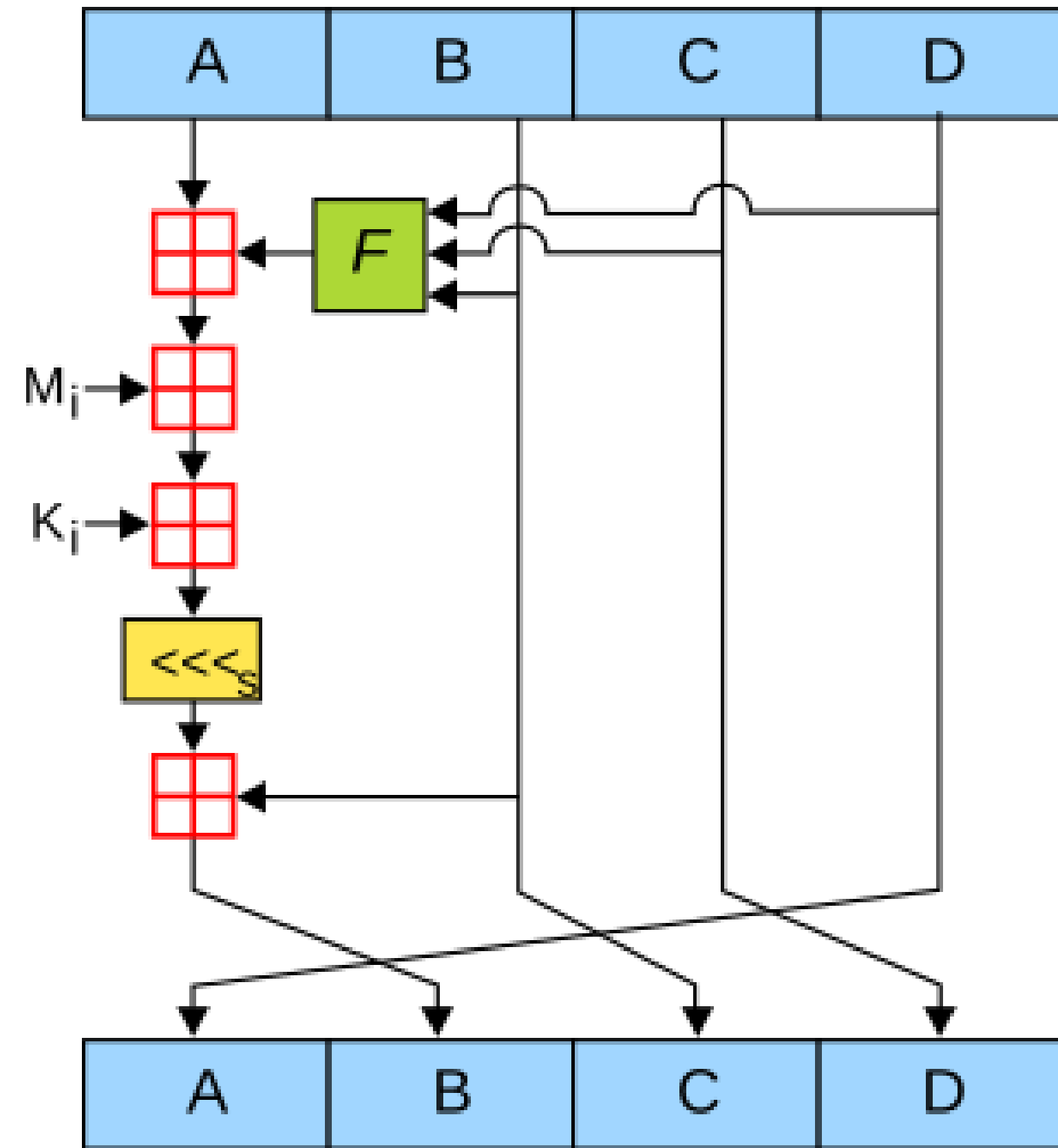
Pass 1: $F(X, Y, Z) = (X \wedge Y) \vee (\neg X \wedge Z)$ [step 0 to 15]

Pass 2: $G(X, Y, Z) = (X \wedge Z) \vee (Y \wedge \neg Z)$ [step 16 to 31]

Pass 3: $H(X, Y, Z) = X \oplus Y \oplus Z$ [step 32 to 47]

Pass 4: $H(X, Y, Z) = Y \oplus (X \wedge \neg Z)$ [step 48 to 64]

- One operation is illustrated in the figure which basically shows how the function F is applied to the four buffers (A, B, C and D), using message word M_i and constant K_i . The term “<<<s” denotes a binary left shift by s-bits.
- After all rounds have been performed, the buffers A, B, C, D contains the MD5 output starting with lower bit A and ending with higher bit D.



Secure Hash Algorithm (SHA)

- The Secure Hash Algorithm (SHA) is a family of cryptographic hash function developed by the National Security Agency (NSA).
- SHA algorithms are widely used for various security applications, including data integrity verification, digital signatures, password storage, and block chain technology.
- There are different member of SHA family, including SHA, SHA-0, SHA-1, SHA2 & SHA-3.

SHA-1

- SHA-1 is a cryptographic hash function which takes an input and produces the output of 160-bits hash value known as message digest.
- It works for any input message that is less than 2^{64} bits.
- It includes following operations :

1. Message Padding:

- The message is padded so that its length is congruent to 448 , mod 512. A single '1' bit is append to the message and then '0' bits are appended so that the length of bits equals 448 modulo 512.

Message	100000	
---------	--------	--

$$(\text{Message length} + \text{padded bits}) \% 512 = 448$$

2. Append Length :

- A 64-bit representation of b is appended to the result of the previous step. The resulting message has a length that is an exact multiple of 512 bits.
- Divide it to N words as $M_0, M_1, \dots, M_{N-1}$ (N is a multiple of 16).

Message	1000000	64 bits
---------	---------	---------

$$(\text{Message length} + \text{padded bits} + 64 \text{ bits}) \% 512 = 0$$

3. Initialize MD Buffer :

- A 5-word buffer (A, B, C, D & E) are used to compute the message digest. Here each of A, B, C, D & E is a 32-bit register. These are initialized to the following values in hexadecimal, low order byte first:

A: 67 45 23 01

B : ef cd ab 89

C : 98 ba dc fe

D: 10 32 54 76

E: c3 d2 e1 f0

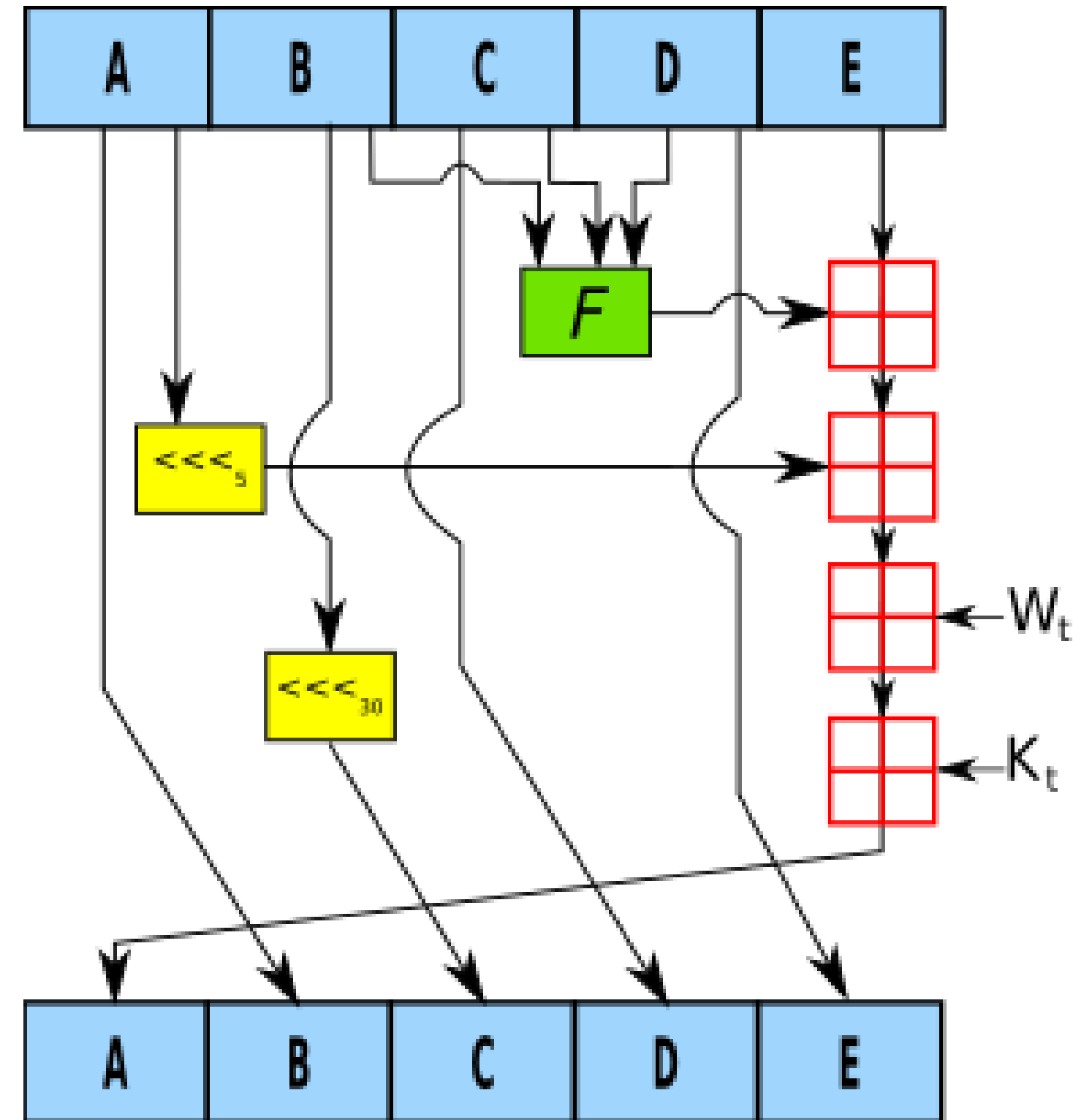
And save as A^0 , B^0 , C^0 , D^0 , E^0

4. Process message in 16-word (512 bit) blocks :

- It contain 4-passes (rounds) with 20 steps or operation each.
- We first define four functions that each take as input three 32-bit words and produce as output one 32-bit word.

$$\begin{aligned} f(t, B, C, D) &= (B \wedge C) \vee (\neg B \wedge D) & \text{if } 0 \leq t \leq 19 \\ f(t, B, C, D) &= B \oplus C \oplus D & \text{if } 20 \leq t \leq 39 \\ f(t, B, C, D) &= (B \wedge C) \vee (B \wedge D) \vee (C \wedge D) & \text{if } 40 \leq t \leq 59 \\ f(t, B, C, D) &= B \oplus C \oplus D & \text{if } 60 \leq t \leq 79 \end{aligned}$$

- One operation is illustrated in the figure which basically shows how the function F is applied, using message word W_t and constant K_t .
- Here $W_t = W_{t-3} \oplus W_{t-8} \oplus W_{t-14} \oplus W_{t-16}$



4. Process message in 16-word (512 bit) blocks :

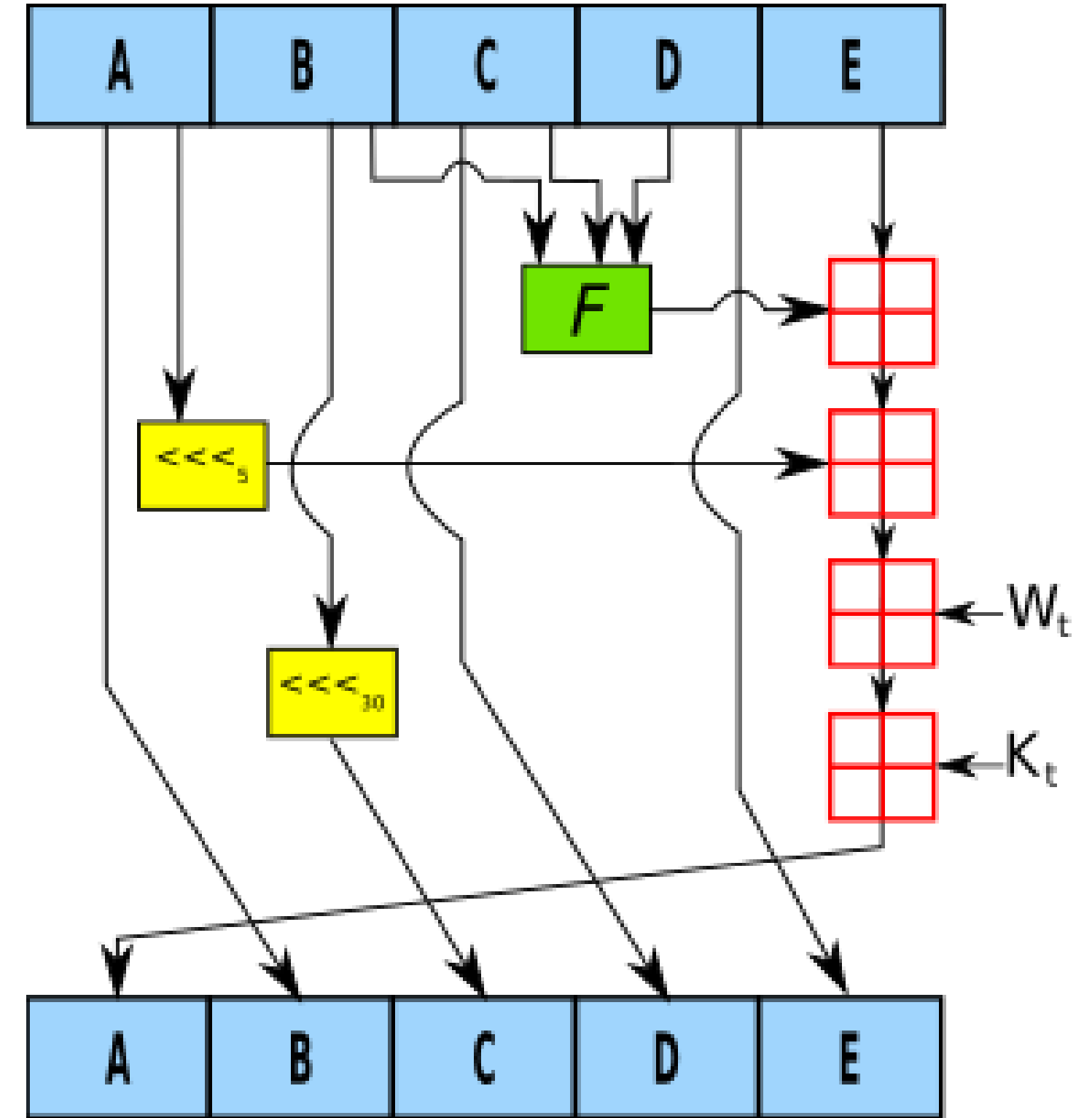
- K_t is the constant depending upon the value of t given by following relations :

$$K_t = 5a827999 \quad \text{if } 0 \leq t \leq 19$$

$$K_t = 6ed9eba1 \quad \text{if } 20 \leq t \leq 39$$

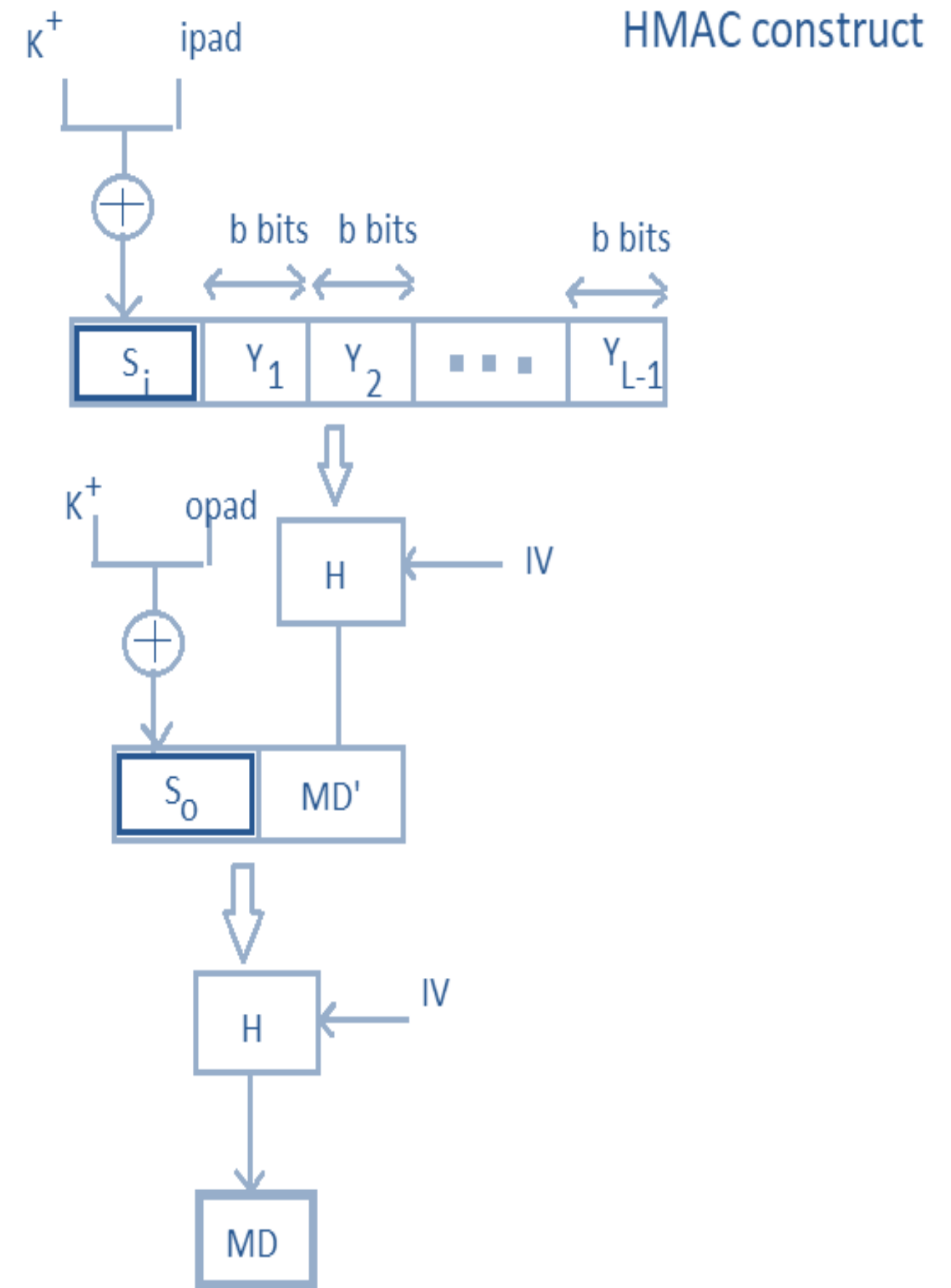
$$K_t = 8f1bbcdc \quad \text{if } 40 \leq t \leq 59$$

$$K_t = ca62c1d6 \quad \text{if } 60 \leq t \leq 79$$



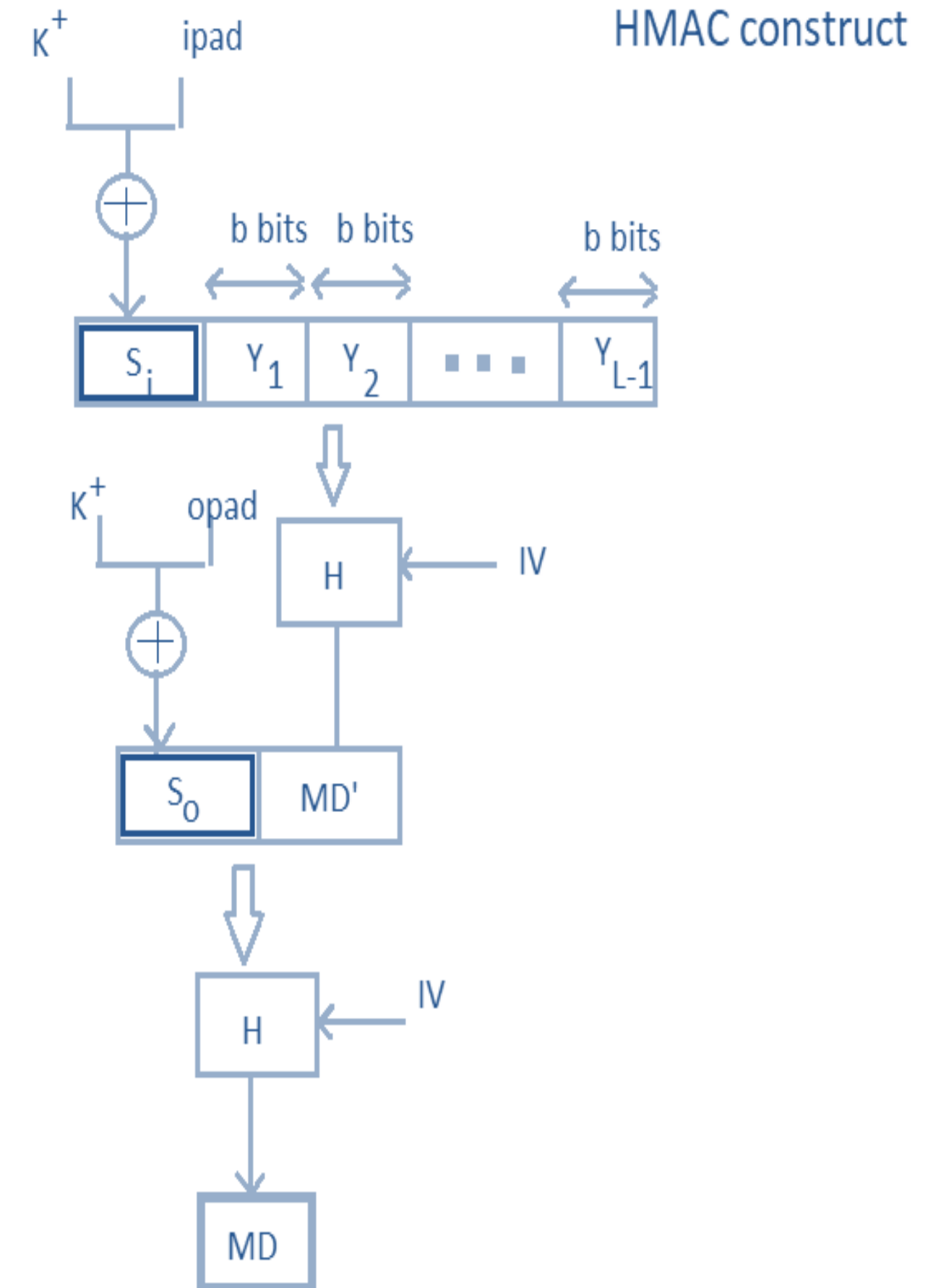
HMAC

- HMAC stands for Hash Based Message Authentication Code. HMAC is a type of MAC which is created by applying a cryptographic hash function to the data and a private shared key. We can use it for data integrity and authentication like any other MAC.
- The Working of HMAC can be explained in following steps.
 1. The secret key is established between sender and receiver. The key length is determined by the underlying hash function used in HMAC. If the key length exceeds the block size of the hash function, it is hashed to match the block size. If the key length is smaller than the block size, it is padded to match the block size.



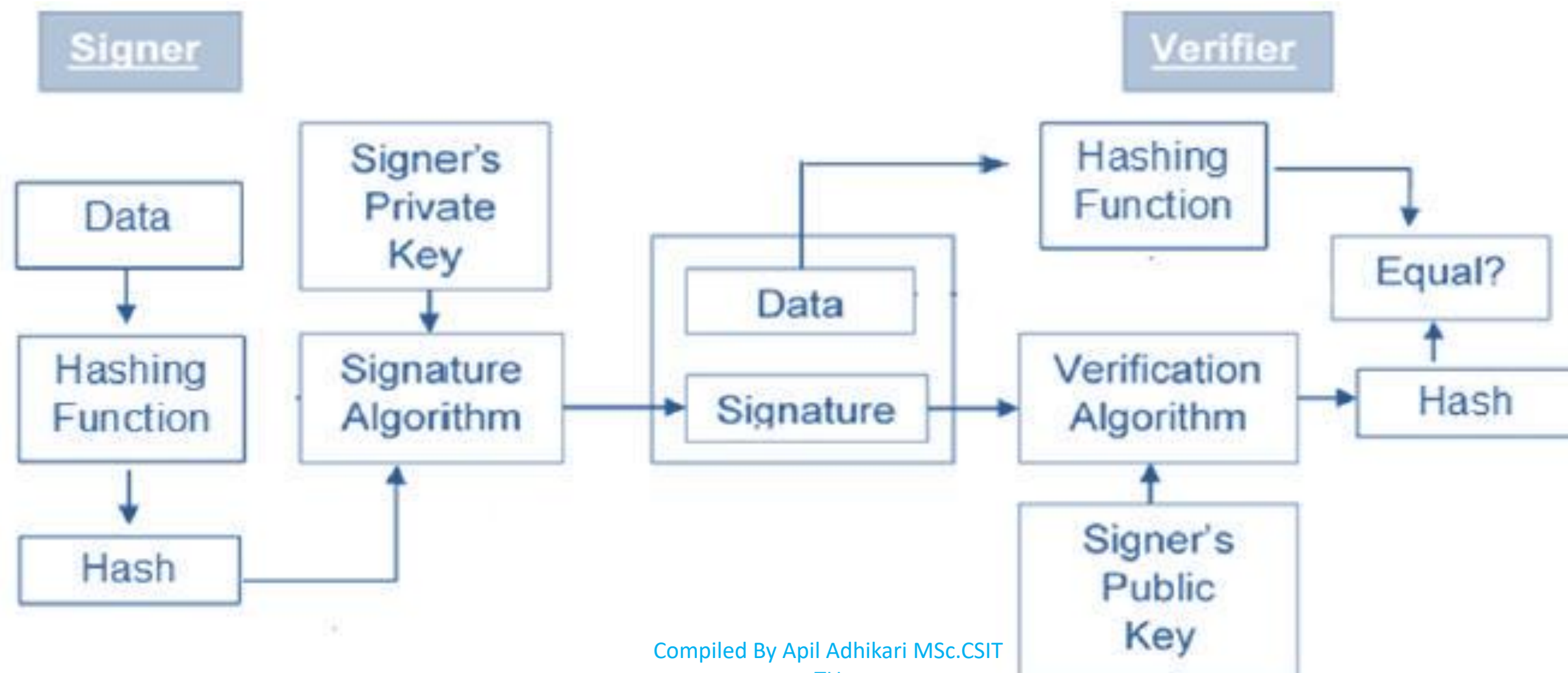
HMAC

2. The secret key is X-ORed with a specific value i.e. $\text{ipad} = 0x\ 36$ Repeated for each block.
3. The result of this inner padding is concatenated with the input message and processed through the hash function. This generates intermediate hash value.
4. The secret key is again X-ORed with different specific value(i.e $\text{opad} = 0x\ 5C$) repeated for each block. It is known as a outer padding.
5. The result of outer padding is concatenated with the intermediate hash value obtained from the previous step and processed through the hash function. This generates the final HMAC.



Digital Signature

- Digital signatures are a cryptographic technique used to provide authentication, integrity, and non-repudiation of digital messages or documents.
- Digital signature is a technique to bind a person/entity to the digital data. This binding can be independently verified by receiver as well as third party.
- Digital signature scheme is based on public key cryptography. The model of digital signature scheme is depicted in the following figure



Digital Signature

- Each person adopting this scheme has a public and private key pair. Generally the key pair used for encryption/decryption and signing verifying are different. The private key is used for signing, is referred to as a signature key and public key as the verification key.
- Signer feeds data to the hash function and generate hash code of data. Hash value and the signature key are then fed to the signature algorithm which produces the digital signature on given hash. Signature is appended to the data and then both are sent to the verifier.
- Verifier fed the digital signature and verification key in to the verification algorithm. The verification algorithm give some value as an output. Verifier also run same hash function on received data to generate hash value. For verification, this hash value and output of verification algorithm are compared and verifier decides whether the digital signature is valid.

Direct and Arbitrated Digital Signature

1. Direct Digital Signature

- A direct digital signature involves only two parties, a sender and a receiver. It is assumed that the receiver knows the public key of the sender. A digital signature may be formed by encrypting the entire message with the sender's private key or by encrypting a hash code of the message with the sender's private key.
- There is a threat of forged digital signatures in direct digital signature.

2. Arbitrated Digital Signature

- An arbitrated digital signature involves three parties, a sender, a receiver and a trusted arbiter. Every signed message from a sender X to receiver Y goes first to an arbiter A, who subjects the message and its signature to a number of tests to check its origin and content. The message is then dated and sent to Y with an indication that it has been verified to the satisfaction of the arbiter.
- There is no threat of forged signatures because an independent arbiter verifies the entire process with due dating and time-stamping.

!!!Thank you!!!